

Vasuki: A Real-Time Snake Game Engine Demonstrating Core Operating System Concepts Through Custom System-Level Libraries in C

Rachit Kumar
Roll No. 230128
*Screen Rendering, Game Logic,
Frontend (WebAssembly)*
rachit-k12@github

Harshal Nerpagar
Roll No. 230076
*Custom Math Operations,
String Handling, File I/O*
HarshalNerpagar@github

Raman Luhach
Roll No. 230107
*Memory Management, Keyboard
I/O, Audio, Random, Timer*
Raman-Luhach@github

Abstract—This paper presents Vasuki, a real-time Snake game engine implemented entirely in C without reliance on the C standard library for core operations. The project replaces nine standard library modules—including `malloc/free`, `strlen/strcpy`, `abs/div`, `fopen/fwrite`, and `rand`—with custom implementations built directly on POSIX system calls and bit-manipulation algorithms. The system demonstrates five fundamental operating system concepts: *memory management* via a 64 KiB first-fit heap allocator with block splitting and coalescing; *process management* via `fork/exec` for asynchronous audio playback with zombie prevention; *I/O management* via non-blocking terminal input using `termios` raw mode and ANSI escape code rendering; *file system operations* via raw `open/read/write/close` system calls; and *security and error handling* through defensive NULL validation, bounds checking, and graceful signal prevention. The engine compiles to both native terminal executables and WebAssembly for browser deployment, totalling 3,857 lines of code across 19 commits. This paper details the architecture, algorithms, system call interfaces, and design trade-offs of each subsystem.

Index Terms—operating systems, memory management, process management, I/O management, file systems, system calls, POSIX, terminal I/O, WebAssembly, game engine

I. INTRODUCTION

Operating system concepts such as memory allocation, process scheduling, and device I/O are typically studied in isolation through textbook exercises. This project takes an alternative approach: implementing a complete, interactive application—a real-time Snake game—where every subsystem is built from scratch using only POSIX system calls and hardware-level algorithms, thereby requiring direct engagement with the OS kernel interface.

The project, named **Vasuki** (Sanskrit for “The Serpent King”), is structured as a layered architecture where nine custom libraries replace their standard C counterparts. The game runs at 10–25 frames per second in a terminal using ANSI escape codes for rendering, and has been cross-compiled to WebAssembly for browser deployment using Emscripten.

The contributions of this work are:

- 1) A **custom heap allocator** implementing first-fit allocation with block splitting and adjacent-block coalescing over a 64 KiB static memory pool.
- 2) **Non-blocking I/O** for real-time keyboard input using `termios` raw mode with `VMIN=0`, `VTIME=0`.
- 3) **Asynchronous process management** using `fork/exec` for audio playback with `waitpid(WNOHANG)` for zombie process prevention.
- 4) **Raw file system operations** using `open/read/write/close` system calls with UNIX permission model.
- 5) **Comprehensive error handling** including division-by-zero protection, NULL pointer validation, memory bounds checking, and terminal state restoration via `atexit`.
- 6) **Cross-platform compilation** to WebAssembly, demonstrating platform abstraction of OS-specific I/O layers.

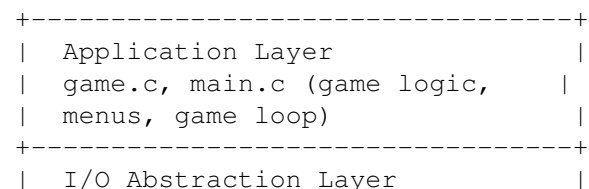
A. Team Contributions

Each team member implemented, tested, and maintains version history for their assigned modules on GitHub:

- **Rachit Kumar (230128)**: `screen.c`, `game.c`, `main.c`, `frontend` (`game.js`, `index.html`, `style.css`, WASM bridge)
- **Harshal Nerpagar (230076)**: `math.c`, `string.c`, `fileio.c`
- **Raman Luhach (230107)**: `memory.c`, `keyboard.c`, `audio.c`, `random.c`, `timer.c`

II. SYSTEM ARCHITECTURE

The system follows a layered architecture with strict dependency ordering:



screen.c	keyboard.c	
audio.c	timer.c	
+	-----+	
Core Libraries Layer		
math.c	string.c	memory.c
random.c	fileio.c	
+	-----+	
OS Kernel Interface		
System calls: tcgetattr,		
tcsetattr, read, write, open,		
close, fork, exec, waitpid,		
usleep, gettimeofday, atexit		
+	-----+	

The core libraries have zero inter-dependencies. The I/O layer depends on core libraries. The application layer depends on both. This ensures testability and clean separation of concerns.

Table I summarizes the nine custom modules.

TABLE I
MODULE SUMMARY

Module	Replaces	LOC
memory.c	malloc/free	104
math.c	<math.h>, abs	149
string.c	<string.h>, sprintf	210
keyboard.c	<ncurses.h>	88
screen.c	<ncurses.h>	113
fileio.c	<stdio.h> file ops	87
audio.c	External audio libraries	34
random.c	rand/srand	43
timer.c	<time.h>	29
Total (libraries)		857
game.c	Game logic	736
main.c	Entry point, menus	460
Frontend (JS/HTML/CSS)	Browser rendering	1,182
Grand Total		3,857

III. METHODOLOGY

A. Memory Management (*memory.c*)

The custom heap allocator manages a 64 KiB static array (`memory_pool[65536]`) divided into blocks, each preceded by a `BlockHeader`:

```

1 typedef struct BlockHeader {
2     int size;           // payload bytes
3     int is_free;        // 1=free, 0=used
4     struct BlockHeader *next;
5 } BlockHeader;
```

The headers form a singly-linked list enabling traversal of the entire pool.

1) *Initialization*: `mem_init()` places a single `BlockHeader` at the start of the pool with `size = MEMORY_POOL_SIZE - sizeof(BlockHeader)`, `is_free = 1`, and `next = NULL`. This creates one contiguous free block spanning the entire pool.

2) Allocation: First-Fit with Block Splitting:

`mem_alloc(size)` traverses the linked list and selects the *first* free block satisfying `block->size ≥ size`. If the remaining space after allocation exceeds `sizeof(BlockHeader) + 16` bytes, the block is *split*: a new free block is created from the residual space and linked into the list.

The first-fit strategy has time complexity $O(n)$ where n is the number of blocks. It was chosen over best-fit ($O(n)$ with full scan, creates small fragments) and worst-fit ($O(n)$, wastes large blocks) as the optimal trade-off between speed and fragmentation for our workload.

3) *Deallocation: Coalescing*: `mem_free(ptr)` computes the header address as `ptr - sizeof(BlockHeader)`, validates that it falls within pool bounds (security check), marks the block as free, and performs a *forward coalescing pass*: adjacent free blocks are merged by absorbing the successor's size plus header into the predecessor:

```
current->size += sizeof(BlockHeader) + current->next->size;
```

(1)

This prevents *external fragmentation*—the condition where sufficient total free memory exists but is scattered in blocks too small to satisfy requests. The coalescing loop does not advance after a merge, allowing chain-merging of three or more consecutive free blocks in a single pass.

4) *Comparison with Production Allocators*: Real-world allocators such as `dldmalloc` use `sbrk()` or `mmap()` to request pages from the kernel. Our allocator uses a pre-allocated static array but implements identical algorithms. Production allocators additionally handle thread safety (via mutexes), memory alignment, and segregated free lists for $O(1)$ average-case allocation [1].

B. I/O Management

I/O management is split across three modules: keyboard input, screen output, and audio output.

1) *Non-Blocking Keyboard Input* (*keyboard.c*): Terminals default to *canonical mode*: input is line-buffered (delivered to the program only after Enter), characters are echoed, and special keys (Ctrl+C) are processed by the kernel's line discipline. For a real-time game, we require *raw mode*, configured via POSIX `termios`:

```

raw.c_lflag &= ~(ICANON | ECHO);
raw.c_cc[VMIN] = 0;
raw.c_cc[VTIME] = 0;
tcsetattr(STDIN_FILENO, TCSANOW, &raw);
```

Clearing `ICANON` disables line buffering, delivering each keypress immediately. Clearing `ECHO` prevents keypresses from appearing on screen. Setting `VMIN = 0`, `VTIME = 0` makes `read()` return immediately with zero bytes if no input is available—this is *non-blocking I/O*, essential for maintaining a consistent frame rate regardless of user input.

Arrow keys are decoded as 3-byte ANSI escape sequences: ESC (0x1B), [(0x5B), followed by A–D for Up, Down,

Right, Left respectively. The `keyboard_pressed()` function reads up to 3 bytes and pattern-matches the sequence.

Terminal restoration is registered with `atexit(keyboard_restore)` to ensure the terminal returns to canonical mode even on unexpected exit paths.

2) *Screen Rendering via ANSI Escape Codes* (`screen.c`): Instead of using libraries like `ncurses`, we communicate directly with the terminal using ANSI X3.64 escape sequences [2]. Key sequences include:

TABLE II
ANSI ESCAPE CODES USED

Sequence	Effect
\033[2J	Clear entire screen
\033[y;xH	Move cursor to row <i>y</i> , column <i>x</i>
\033[?25l	Hide cursor
\033[32m	Set text color to green
\033[0m	Reset all formatting

Rendering uses *differential updates*: only changed cells (snake head, erased tail, food) are redrawn each frame, avoiding full-screen clears that cause visible flickering. This is analogous to how OS display compositors minimize repaints.

The `screen_flush()` function calls `fflush(stdout)` to force the output buffer to the terminal. Without explicit flushing, `stdout` buffering causes visible frame lag—a direct demonstration of OS-level I/O buffering semantics.

3) *Audio via Process Creation* (`audio.c`): See Section III-C for the `fork/exec` audio implementation.

C. Process Management

The audio subsystem uses UNIX process creation to play sound files without blocking the game loop:

```

1 void audio_play(const char *path) {
2     pid_t pid = fork();
3     if (pid == 0) { // Child
4         close(STDERR_FILENO);
5         execlp("afplay", "afplay",
6             path, (char *)0);
7         _exit(1);
8     }
9     if (pid > 0) // Parent
10        waitpid(-1, NULL, WNOHANG);
11 }

```

`fork()` creates a child process that is an exact copy of the parent. The child calls `execlp()` to replace its program image with the `afplay` audio player. The parent returns immediately and continues the game loop—the sound plays asynchronously in the child process.

1) *Zombie Prevention*: When a child exits, its process table entry persists as a *zombie* until the parent reads its exit status. `waitpid(-1, NULL, WNOHANG)` performs a non-blocking check, reaping any finished children without stalling the game. The `WNOHANG` flag is essential: without it, `waitpid` would block until a child terminates, freezing the game.

2) *_exit vs exit*: The child uses `_exit(1)` rather than `exit(1)` after a failed `exec`. `exit()` runs `atexit` handlers (which would prematurely restore the terminal) and flushes `stdio` buffers (which could corrupt the parent’s output). `_exit()` bypasses both, performing only the kernel-level process termination.

D. File System Operations (`fileio.c`)

The file I/O module wraps raw POSIX system calls, bypassing the C standard `stdio` buffered layer (`fopen/fread/fwrite`):

```

1 int file_open_write(const char *path) {
2     return open(path,
3         O_WRONLY | O_CREAT | O_TRUNC,
4         0644);
5 }

```

1) *System Call Interface*: Four system calls are used directly:

- `open(path, flags, mode)`: Returns an integer *file descriptor*—an index into the process’s file descriptor table maintained by the kernel.
- `read(fd, buf, count)`: Reads up to `count` bytes into `buf`. Returns the number of bytes actually read.
- `write(fd, buf, count)`: Writes `count` bytes from `buf` to the file.
- `close(fd)`: Releases the file descriptor.

2) *UNIX Permission Model*: Files are created with permission mode 0644 (octal), encoding the standard UNIX `rw-rw-rw-` triplet:

$$0644 = \underbrace{110}_{rw-} \underbrace{100}_{r--} \underbrace{100}_{r--} \quad (2)$$

Owner has read and write access; group and others have read-only access.

3) *High Score Persistence*: The helper functions `file_save_int()` and `file_load_int()` serialize integers to ASCII strings (using `int_to_string()` from `string.c`) before writing, and parse them back (using `string_to_int()`) after reading. This demonstrates the complete lifecycle: `open` → `write` → `close` for saving, `open` → `read` → `close` for loading.

4) *Device File I/O*: The random module (`random.c`) reads from `/dev/urandom`, a kernel-provided device file that generates cryptographically random bytes from the OS entropy pool. The same `open/read/close` system calls are used, demonstrating the UNIX “everything is a file” philosophy—device I/O and file I/O share an identical interface.

E. Security and Error Handling

Defensive programming is applied throughout the system to prevent crashes, undefined behavior, and resource leaks:

1) Signal Prevention:

- **SIGFPE prevention:** `custom_divide()` and `custom_modulo()` return 0 when the divisor is zero, preventing the kernel from delivering a fatal SIGFPE signal.
- **SIGSEGV prevention:** All nine string functions validate input pointers against NULL before dereferencing. `mem_free()` validates that the pointer falls within the memory pool bounds.

2) **Buffer Overflow Prevention:** `strncpy()` accepts a maximum length parameter and always null-terminates the destination buffer, preventing the buffer overflow vulnerabilities inherent in `strcpy()`. The snake body uses a fixed-size array with `MAX_SNAKE_LENGTH = 256` and growth is checked against this bound.

3) Resource Leak Prevention:

- `atexit(keyboard_restore)` ensures terminal settings are restored even on unexpected exit paths.
- File descriptors are checked (`fd >= 0`) before operations and closed after use.
- The memory allocator's `mem_print_debug()` allows runtime verification of allocation state.
- Zombie processes are prevented via `waitpid(WNOHANG)` after every `fork()`.

4) Graceful Degradation:

- If `/dev/urandom` is unavailable, the PRNG falls back to a default seed.
- If `execvp()` fails (audio player not installed), the child exits silently; the game continues without sound.
- If food placement fails after 200 random attempts, a deterministic fallback position is used.
- If `mem_alloc()` cannot find a fitting block, it returns NULL.

F. Custom Arithmetic via Bit Manipulation (`math.c`)

All arithmetic operations are implemented using bitwise operators, mirroring hardware ALU behavior:

1) **Shift-and-Add Multiplication:** Binary multiplication processes each bit of the multiplier b . For each set bit at position i , the multiplicand a shifted left by i is added to the result:

$$a \times b = \sum_{i=0}^{31} (b_i \cdot (a \ll i)) \quad (3)$$

where b_i denotes the i -th bit of b . Time complexity is $O(\log b)$.

2) **Binary Long Division:** Division finds the highest bit position k such that $b \ll k \leq a$, then iteratively subtracts $b \ll i$ from a for each valid position i from k down to 0, setting the corresponding bit in the quotient.

3) Branchless Absolute Value:

$$|x| = (x \oplus m) - m, \quad \text{where } m = x \gg 31 \quad (4)$$

The arithmetic right shift by 31 produces a mask of all 1s (i.e., -1) for negative values and all 0s for non-negative

values. XOR with the mask produces the one's complement for negatives, and subtracting m (adding 1) yields the two's complement negation. This avoids branch prediction penalties in the CPU pipeline.

G. Pseudo-Random Number Generation (`random.c`)

The PRNG uses the `xorshift32` algorithm [3]:

```
static unsigned int xorshift(void) {
    rng_state ^= rng_state << 13;
    rng_state ^= rng_state >> 17;
    rng_state ^= rng_state << 5;
    return rng_state;
}
```

The generator has a period of $2^{32} - 1$ and passes standard statistical randomness tests. It is seeded from `/dev/urandom` at initialization, providing non-deterministic game behavior across sessions.

H. Cross-Platform Compilation to WebAssembly

The game logic compiles unchanged to WebAssembly via Emscripten. A bridge module (`wasm_bridge.c`, 196 LOC) exports 36 C functions to JavaScript. Platform-specific I/O is replaced at link time:

TABLE III
PLATFORM ABSTRACTION

Subsystem	Native	Browser
Rendering	ANSI escape codes	HTML5 Canvas
Input	termios raw mode	addEventListener
Audio	fork+exec	Web Audio API
Timing	gettimeofday	emscripten_get_now
File I/O	open/write	localStorage
Random	/dev/urandom	emscripten_random

IV. IMPLEMENTATION RESULTS

The game supports two modes (Endless and 5-Level), five arena layouts with distinct obstacle configurations, a combo scoring system (up to $5 \times$ multiplier for consecutive food collection within 3 seconds), bonus food with timed expiration, dynamic speed acceleration, and persistent high scores.

TABLE IV
BUILD CONFIGURATION

Parameter	Value
Compiler	cc (C99, -Wall -Wextra -Werror)
WASM Compiler	emcc (-O2, MODULARIZE)
Total LOC	3,857
Git Commits	19 (across 3 contributors)
Frame Rate	10 FPS (100 ms) to 25 FPS (40 ms)
Memory Pool	64 KiB static allocation
Max Snake Length	256 segments
Play Area	60 $\times$ 20 cells

V. FUTURE SCOPE

Several directions for future enhancement exist:

- 1) **Networked Multiplayer:** Implementing a client-server architecture using UNIX sockets (`socket/bind/accept/connect`) for real-time multiplayer gameplay, introducing concepts of inter-process communication, synchronization, and network I/O.
- 2) **Multi-threaded Architecture:** Separating input handling, game logic, and rendering into distinct POSIX threads (`pthread_create`), requiring mutexes and condition variables for shared state protection—demonstrating concurrency and synchronization.
- 3) **Dynamic Terminal Adaptation:** Handling the `SIGWINCH` signal (delivered on terminal resize) to dynamically adjust the game area, demonstrating signal handling and asynchronous event processing.
- 4) **Virtual Memory Simulation:** Extending the memory allocator to support paging with a page table, simulating virtual-to-physical address translation and page replacement algorithms (LRU, FIFO).
- 5) **Filesystem-based Replay:** Recording game sessions to files and implementing a replay system, demonstrating sequential file I/O and serialization/deserialization.
- 6) **Shell Integration:** Building a custom mini-shell that launches the game as a child process, supporting job control (`SIGTSTP/SIGCONT`) to suspend and resume the game.

VI. CONCLUSION

Vasuki demonstrates that core operating system concepts—memory management, process management, I/O management, file systems, and error handling—are not abstract textbook constructs but practical engineering requirements encountered when building real interactive software from scratch. By replacing standard library abstractions with custom implementations built directly on system calls, the project provides concrete, executable understanding of the OS kernel interface. The successful cross-compilation to WebAssembly further validates the architectural separation between platform-independent logic and platform-specific I/O, a principle fundamental to operating system design.

REFERENCES

- [1] D. Lea, “A Memory Allocator,” *Unix/Mail*, 1996. [Online]. Available: <http://gee.cs.oswego.edu/dl/html/malloc.html>
- [2] ECMA International, “Control Functions for Coded Character Sets,” Standard ECMA-48, 5th ed., 1991.
- [3] G. Marsaglia, “Xorshift RNGs,” *Journal of Statistical Software*, vol. 8, no. 14, pp. 1–6, 2003.
- [4] M. Kerrisk, *The Linux Programming Interface*. San Francisco, CA: No Starch Press, 2010.
- [5] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Upper Saddle River, NJ: Pearson, 2014.
- [6] W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed. London, UK: Pearson, 2018.
- [7] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: Wiley, 2018.
- [8] A. Zakai, “Emscripten: An LLVM-to-JavaScript Compiler,” in *Proc. ACM Int. Conf. Object Oriented Programming Systems Languages and Applications (OOPSLA)*, 2011, pp. 301–312.